

基于大语言模型的思维链推理策略对比研究

Self-Consistency、Step-Aware Verifier、RAG + CoT、Multi-Agent Debate 与 Harness Engineering

nlp-cot 项目组

浙江大学 自然语言处理课程大作业

2026 年 6 月

- 1 任务背景
- 2 Self-Consistency
- 3 Step-Aware Verifier
- 4 RAG + CoT
- 5 Multi-Agent Debate
- 6 Bonus: Harness Engineering
- 7 实验结果
- 8 总结



选题要求

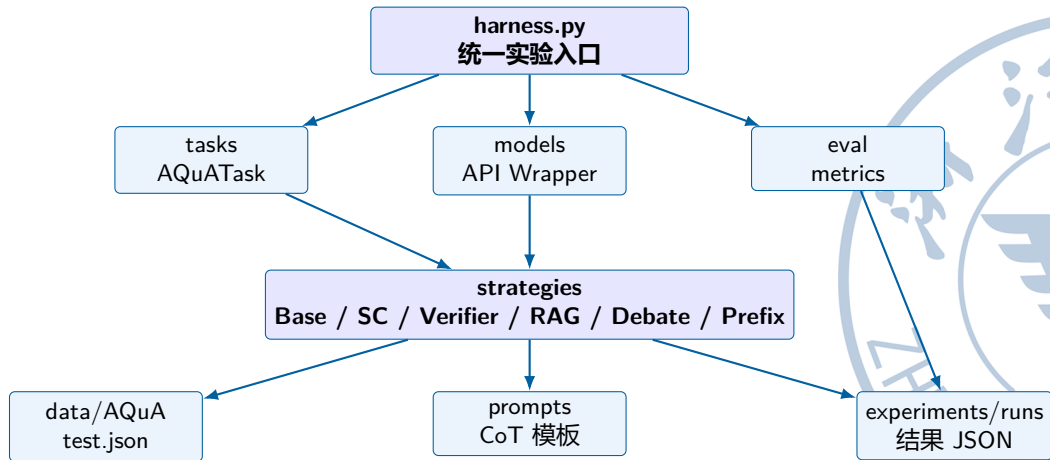
- 面向 AQUA 等推理数据集，分析大语言模型的 Chain-of-Thought 能力。
- 复现并比较多路径推理、verifier、检索增强、多 Agent 协作等方法。
- 关注准确率、推理成本、工程可复现性与失败案例。
- Bonus：用 harness engineering 管理 agentic 推理任务。

核心问题

单条 CoT 容易受采样随机性、答案格式、局部计算错误和自相矛盾影响。

本项目目标

在统一实验框架下，把不同策略的收益与成本放在同一把尺子上比较。



关键代码：harness.py 负责加载任务、模型、策略并保存实验结果。

论文核心思想

- 用较高温度采样多条 CoT 推理路径.
- 每条路径给出一个最终选项.
- 通过聚合答案降低单次采样偶然错误.

原始多数投票

路径数多的答案获胜:
 $\arg \max_y \sum_i \mathbb{1}[\hat{y}_i = y]$

项目中的改进点

多数票不一定代表高质量推理. 因此我们负责的 self-consistency 部分改为:

$$\arg \max_y \sum_i q_i \cdot \mathbb{1}[\hat{y}_i = y]$$

其中 q_i 是第 i 条路径的启发式质量分.

质量分由什么决定

信号	作用
答案清晰	是否严格输出 Answer: X
推理步骤	是否有显式 step 或分段推理
数学痕迹	是否包含数字、运算符、等式等计算信号
输出长度	过短扣分, 过长轻微扣分
自相矛盾	多处答案不一致时扣分

一个直观例子

答案	路径数	加权票
A	2	$1.8 + 1.7 = 3.5$
B	3	$0.8 + 0.7 + 0.6 = 2.1$

旧算法选 B, 因为 B 有 3 票; 新算法选 A, 因为 A 的两条推理更清晰、更完整.

边界

该方法只使用本路径文本的局部信号, 不调用 verifier、不检索外部知识, 也不引入多 Agent.

关键函数

- `_path_quality`: 计算每条路径的启发式质量分.
- `_aggregate_votes`: 按权重聚合, 并用原始票数、最佳路径质量、出现顺序打破平票.
- `_can_stop_early`: 当领先答案无法被剩余路径追上时提前停止.
- `run`: 生成路径、抽取答案、重试空答案、聚合最终预测.

伪代码

```
for i in range(n_paths):  
    raw = model.generate(prompt, temperature)  
    pred = task.extract_answer(raw)  
    if retry_on_empty and not pred:  
        raw = model.generate(prompt, low_temp)  
    weight = _path_quality(raw, pred)  
    save(pred, weight)  
    if _can_stop_early(...):  
        break  
  
final = _aggregate_votes(predictions, weights)
```

低温重试让模型输出更稳定, 主要用于补救没有 Answer: X 的样本.

论文思想

- 把推理过程拆成 step.
- 对每一步进行正确性或可信度评估.
- 用 verifier 选择更可靠的推理路径.

项目实施

- 策略入口:
strategies/step_verifier.py.
- verifier 模型封装:
models/deberta_verifier.py.
- 结果保存 step 级评分, 用于选择最终答案.

实验观察

AQuA 前 100 条达到 94%, 但平均输出 token 与耗时明显高于 base/self-consistency.

论文思想

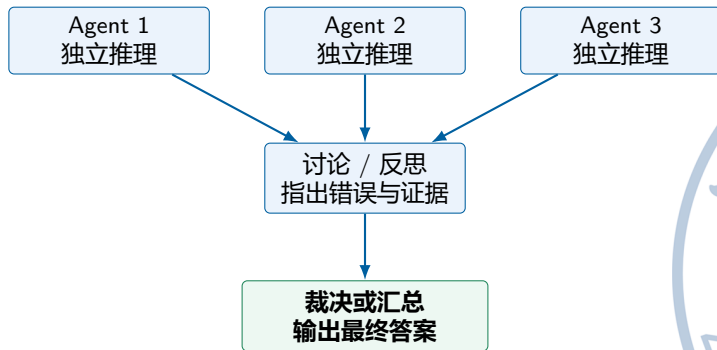
- 在推理过程中穿插检索.
- 把 retrieved evidence 与 CoT 推理交替组织.
- 适合知识密集型、多跳问答.

项目实现与局限

- 策略入口: `strategies/rag_cot.py`.
- 对 AQuA 这类数学题, 检索信息不总是关键瓶颈.
- 输入 token 上升明显; 收益依赖检索质量与任务类型.

实验观察

100 样本准确率 92%, 高于 base 但低于 self-consistency 与 verifier.



项目实施

strategies/multi_agent_debate.py 中模拟多角色生成、互评与最终汇总。

实验观察

100 样本准确率 95%，但真实 API 成本高于保存结果中展示的最终摘要 token。

为什么需要 harness

- 多策略、多参数、多 API 后端很容易变成不可复现实验.
- Harness 把任务、模型、策略、指标、结果保存拆成稳定接口.
- 便于替换个人负责的 `strategies/*.py`, 不影响其他同学模块.

子系统覆盖

策略	指令	环境	状态	反馈
Base	是	是	否	否
SC	是	是	是	否
Verifier	是	是	是	是
RAG	是	是	是	部分
Debate	是	是	是	是

数据集

AQuA

test split 前 100 条

模型接口

API

DMXAPI / DeepSeek 兼容格式

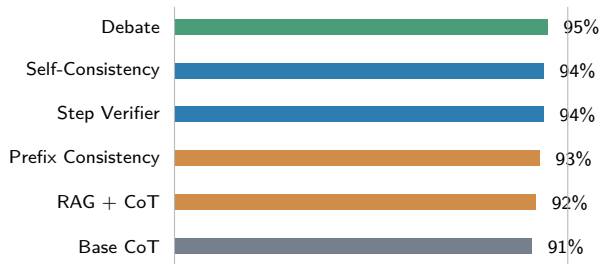
评估指标

Accuracy

同时记录 token、步数、耗时

运行方式示例

```
python harness.py --strategy self_consistency --dataset aqua --n_samples 100  
--n_paths 7
```



结论

- Base CoT 已有 91% 基线.
- Self-Consistency 提升到 94%, 收益稳定且工程侵入小.
- Debate 最高为 95%, 但成本与调度复杂度更高.

策略	样本	正确	准确率	平均输出	观察
Base CoT	100	91	91%	187.64	单路径，成本最低.
Self-Consistency	100	94	94%	238.56	多路径采样，准确率稳定提升.
Step Verifier	100	94	94%	563.74	step 级评估更细，但输出成本最高.
RAG + CoT	100	92	92%	197.90	输入 token 增加，受检索质量影响.
Multi-Agent Debate	100	95	95%	72.18	结果文件只保存摘要，实际调用成本更高.

单位为实验 JSON 记录的平均 output tokens; Debate 的记录低估了真实中间讨论成本.

策略	前 50 条	前 100 条
Multi-Agent Debate	94%	95%
Self-Consistency	94%	94%
Step Verifier	92%	94%
Prefix Consistency	94%	93%
RAG + CoT	78%	92%
Base CoT	92%	91%

分析

- Self-Consistency 在 50/100 样本上均为 94%，波动最小.
- RAG 前 50 条偏低，对样本分布和检索质量更敏感.
- Debate 准确率最高，但更适合作为高成本上限方案.

方法	主要解决的问题	优势	局限
Self-Consistency	单路径随机错误	简单、低风险、可直接替换策略文件	多次采样增加调用成本
Step Verifier	中间步骤错误	可解释性强，能定位 step 级问题	verifier 依赖与耗时较高
RAG + CoT	外部知识不足	对知识密集题更有潜力	数学题收益受检索质量限制
Debate	单模型视角偏差	多角色互相纠错，准确率最高	调度复杂、成本最高
Harness	实验不可复现	统一接口、结果留痕、便于协作	需要规范每个策略的输入输出

算法贡献

- 将普通 self-consistency 从 “多数投票” 改为 “自适应加权投票”。
- 加入答案有效性检查、空答案低温重试、早停剪枝和平票质量打破。
- 不占用 verifier、RAG、debate 的方法边界，只用本路径文本启发式信号。

工程贡献

- 统一策略接口，方便各成员替换 strategies 下自己的方法。
- 实验结果自动写入 experiments/runs，便于复查与报告。
- 支持 API key / base url / model 参数化，避免硬编码密钥。

核心代码路径

路径	作用
<code>harness.py</code>	统一实验入口与结果保存.
<code>strategies/self_consistency.py</code>	自适应加权 self-consistency.
<code>strategies/step_verifier.py</code>	Step-aware verifier 策略.
<code>strategies/rag_cot.py</code>	检索增强 CoT.
<code>strategies/multi_agent_debate.py</code>	多 Agent debate.
<code>experiments/runs/*.json</code>	实验结果与逐样本输出.

参考文献

Self-Consistency Improves Chain of Thought Reasoning in Language Models; Making Large Language Models Better Reasoners with Step-Aware Verifier; Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions; Improving Factuality and Reasoning in Language Models through Multiagent Debate.

谢谢!

欢迎提问

